What We Added (Tech Stack)

1. Smart Text Understanding (Sentence-BERT)

Before: The system could only find exact matches. If a resume said "Node.js" but the job asked for "Node JS" or "NodeJS", it wouldn't recognize them as the same.

After: We added a **"smart brain"** that understands the *meaning* behind words, not just exact spelling. It's like how a human can recognize that "Node.js", "Node JS", and "NodeJS" all mean the same thing.

---

2. Vector Search (Meaning-Based Search)

Before: The system searched like using Ctrl+F in a document — exact word matches only.

After: We taught it to search by **meaning**. So when we look for candidates, instead of just matching keywords, it finds people whose experience makes *sense* for the job, even if they used different words.

Example: A job asks for "REST APIs" but someone's resume says "web services" → the system now understands these mean similar things and makes the connection.

---

3. Skill Recognition with AI (Deep Learning)

Before: The system had a fixed list of ~130 skills it knew about. Anything else was ignored.

After: We added **AI-powered skill spotting** that can:

- Recognize misspellings (e.g., "Pythn" → "Python")
- Find skills hiding in sentences, not just skill lists
- Understand category-related skills (e.g., "cloud platforms" like AWS, Azure, GCP)

Result: Caught 26 out of 34 spelling variants (vs. only 7 before).

---

4. Gap Analysis (What's Missing)

New feature: Now the system shows:

- ✅ **What skills the candidate HAS** that match the job
- ❌ **What skills are MISSING** (but maybe they have similar ones)
- 📊 **A match percentage** (e.g., "70% match")

This is like a checklist: "Job needs Python, SQL, AWS. Candidate has Python and SQL but missing AWS."

---

5. Smart Resume Summaries

New feature: Instead of showing the entire resume, the system pulls out the **3 most relevant sentences** that match the job description.

Why Does This Matter?

Before	**After**
Miss candidates because of spelling differences	Find candidates even with typos
Can only match exact keywords	Understand meaning and relationships
Slow, keyword-only matching	Fast, intelligent matching
Recruiters see raw resume data	Recruiters see summaries + gap analysis
No understanding of similar skills	Recognizes when someone has a *similar* skill

---

Next Phase DL Plan

The Goal: Prove your system works with real numbers—not just "we used AI," but "we measured it and improved it."

What to Add (In Order of Importance)

MUST-DO (Priority 1-3)

These are mandatory—the evaluators will expect them.

1️⃣ Ranking Benchmark (Nov 3)

What: Right-now you don't have numbers for "how good is the ranking?"
How to do it:

- Take 10 resumes + 3 different job descriptions = 30 resume-job pairs
- Two people (you + a teammate) score each pair: 0 = poor match, 1 = okay, 2 = good, 3 = excellent
- Compare your AI ranking against these real scores
- Measure: How many top 3 candidates did the AI get right? (called "Precision@3")

Why it matters: The biggest question evaluators will ask is "**How accurate is this ranking?**" Right now you have no answer. This gives you one.

2️⃣ Fix Experience Parsing (Nov 3)

What: Remember how Siddharth's resume shows "0 years"? That's a bug—his experience should show correctly.
How to do it:

- Look at the regex that finds "5 years of experience" etc.
- Fix the patterns it matches
- Test on your 10 demo resumes
- Make sure the charts match the actual ranking

Why it matters: It's a visible bug. Fixing it makes the system look more professional.

3️⃣ UI Honesty (Nov 6)

What: The dashboard says "JobBERT Match" but the model is really just a general sentence model, not job-specific.
How to do it:

- Rename "JobBERT Match" → "Semantic Match"
- Add a tooltip: "This score is based on semantic similarity between the resume and the job description"

Why it matters: Honesty. If evaluators click on it and see you're using a general model, you get points for being truthful instead of losing points for exaggeration.

STRETCH GOALS (Priority 4-6)

Do these IF you have time — they are impressive but not required.

4️⃣ Typos & Abbreviations (Nov 4)

What: Right now k8s, sklearn, golang are not recognized.
How to do it:

- Use a fuzzy matching library (e.g., rapidfuzz)
- Create an alias list: k8s → Kubernetes, sklearn → Scikit-learn, etc.
- Test: Can it now recognize these misspellings?

Why it matters: Shows you're thinking about real-world messy data.

5️⃣ Fine-tune the AI Model (Nov 5)

What: Take the pre-trained Sentence-BERT model and train it yourself on skill pairs (e.g., "nodejs" and "Node.js" are the same).
How to do it:

- Create 50-100 pairs of similar/different skills
- Use sentence-transformers library to fine-tune for 2-3 minutes on CPU
- Measure: Does the fine-tuned version recognize more spelling variants than the pre-trained one?

Why it matters: NOW you can say "**We trained a model**" instead of just "we used a pre-trained one."

6️⃣ Bias Check (Nov 6)

What: Swap names (e.g., "Priya" ↔ "John") and gender words in resumes, then re-rank. Does the score change?
How to do it:

- Create versions of your 10 resumes with different names
- Run them through the system
- Measure: Did the ranking change? By how much?

Why it matters: Shows you care about fairness. Evaluators love this.